

TD N° 1 : les expressions, les opérateurs, priorité

Exercice 1 :

1.

-1.0 : constante réelle sous forme décimale de type double

34.5L : constante réelle sous forme décimale de type long double (type forcé)

0xeba : constante entière sous forme hexadécimale de type int, sa valeur décimale : 3786

0123l : constante entière sous forme octale de type long (type forcé) sa valeur décimale : 83

1.23ul : incorrecte les suffixes ul n'existent pas pour les réels

345UL : constante entière sous forme décimale de type unsigned long (type forcé)

0XEUL : constante entière sous forme hexadécimale de type unsigned long, valeur décimale 14

Constante	Type	Valeur décimale
-1.0	double	-1.0
34.5L	long double	34.5
0xeba	int	3786
0123l	long	83
1.23ul	Erreur	N/A
345UL	unsigned long	345
0XEUL	unsigned long	14

2. Tenir compte les valeurs initiales de chaque variable

- $A += (X+5) \Leftrightarrow A += 17 \Leftrightarrow A = A + 17 \Leftrightarrow A = 37$
- $A == (B=5) \Leftrightarrow B = 5$ et l'expression vaut 0
- $A != (C * (-D)) \Leftrightarrow C * (-D) \Leftrightarrow C = C * (-D) \Leftrightarrow C = 20$ et $A != 20$ l'expression vaut 0
- $A \% = D ++ 5 \Leftrightarrow A = A \% D$ et $D = D + 1 \Leftrightarrow A = 20 \% 2 = 0$ et $D = 3$ alors l'expression vaut 0
- $A = X * (B < C) + Y * (B < C) \Leftrightarrow A = X * (5 < -10) + Y * (5 < -10) \Leftrightarrow A = 12 * 0 + 15 * 0$ alors l'expression vaut 0

Tenir compte les valeurs rendues comme résultat de l'expression et les valeurs des variables dont le contenu a changé

- $A += (X+5) \Leftrightarrow A += 17 \Leftrightarrow A = A + 17 \Leftrightarrow A = 37$
- $A == (B=5) \Leftrightarrow B=5$ et l'expression vaut 0
- $A != (C * = (-D)) \Leftrightarrow C * = (-D) \Leftrightarrow C = C * (-D) \Leftrightarrow C = 20$ et $A != 20$ l'expression vaut 1
- $A \% = D++ \ 5 \Leftrightarrow A = A \% D$ et $D = D + 1 \Leftrightarrow A = 37 \% 2 = 1$ et $D = 3$ alors l'expression vaut 1
- $A = X * (B < C) + Y * (B < C) \Leftrightarrow A = X * (5 < 20) + Y * (5 < 20) \Leftrightarrow A = 12 * 1 + 15 * 1$ alors l'expression vaut 27

Exercice 2 :

```
int main() {  
    int x,y,z ;  
  
    x= -3+ 24 %5 *2 ; printf("%d\n", x) ; // on commence par % puis * puis -  $\Leftrightarrow x=5$   
    x *= y= z= 4; printf("%d\n", x) ; // z=4 et y=4 et  $x=x*y = 5*4= 20$   
    x = y== z; printf("%d\n", x) ; // y==z est vrai elle vaut 1 alors x=1  
    x == (y= z); printf("%d\n", x) ; // y=z=5 et x==5 vaut 0 donc la valeur de x sera toujours la  
    dernière valeur retenue est x=1  
}
```

Exercice 3:

L'objectif de l'exercice est de faire attention au type de variable avec le type de spécificateurs de format.

```
main() {  
    double d=3.2, x ;  
    int i=2, y ;  
    x= (y=d/i) *2 ; printf(" x= %f\t", x) ; printf(" y= %f\t", y) ; // y est de type int , il faut utiliser le %d  
    sinon le code ne se compile pas.  
    y= (x=d/i) *2 ; printf(" x= %f\t", x) ; printf(" y= %f\t", y) ;  
    y= d *(x=2.5/d) ; printf(" y= %f\t", y) ;  
    x= d *(y=((int) 2.9+1.1 /d) ; printf(" x= %f\t", x) ; printf(" y= %f\t", y) ; // commencer par / puis  
    l'addition
```

Exercice 4:

```
main() {  
    int x=03, y=02, z=01 ; // x=0011 , y=0010 , z=0001
```

```
printf(" x^y & ~z= %d\n", x^y&~z) ; // x^y&~z on commence par ~z=1110, y&~z= 0010 puis  
x^y&~z=0001
```

```
x=1 ; y=23 ; // x=0001 et y=0001 0111
```

```
x<<=3 ;// on élimine les 3 bits à gauche et les remplies par des 3 bits de 0 à droite
```

```
printf("x=%d\n ",x) ; //x= 00000001 vaut x<<=3 x=0000 1000= 8
```

```
y>>=2 ; // on élimine les 2 bits à droite et les remplies par des 2 bits de 0 à gauche
```

```
printf("y=%d\n ",y) ;// y=0001 0111 vaut y>>=2 y= 00000101
```

Exercice 5 :

1. **y = i / j;**

- i et j sont tous les deux des **entiers**. Lorsqu'une division entière est effectuée en C, le résultat est également un entier (les décimales sont ignorées).
- Expression : $2 / 4 = 0.5$, mais comme c'est une **division entière**, le résultat sera **0** (les parties décimales sont éliminées).
- Ce résultat entier (0) est ensuite stocké dans la variable y de type float. Le contenu de y sera donc **0.0**.

2. **z = (float) i / j;**

- Ici, l'entier i est converti en un flottant avec l'expression (float) i, donc i devient 2.0 avant la division.
- La division est alors effectuée avec un flottant ($2.0 / 4$), ce qui donne **0.5**.
- Le résultat sera **0.5**, et il est stocké dans z (qui est aussi un float).
- Le contenu de z sera donc **0.5**.

Exercice 6 :

```
int n=1,p=6 ;
```

```
int q ;
```

```
float x ;
```

1. **q = n < p;**

- Expression : $n < p \rightarrow 1 < 6$, ce qui est **vrai** (1 en C).
- Résultat : q = 1.
- Valeur de q : **1**.

2. **q = n == p;**

- Expression : $n == p \rightarrow 1 == 6$, ce qui est **faux** (0 en C).
- Résultat : $q = 0$.
- Valeur de q : **0**.

3. **$q = p \% n + p > n$;**

- Le mod (%) calcule le reste de la division :
 - $p \% n = 6 \% 1 = 0$ (puisque tout nombre modulo 1 est 0).
- Expression : $0 + 6 > 1$, soit $6 > 1$, ce qui est **vrai** (1).
- Résultat : $q = 1$.
- Valeur de q : **1**.

4. **$x = p / n$;**

- Division entière : $6 / 1 = 6$.
- Résultat : $x = 6$ (qui est converti en flottant, donc $x = 6.0$).
- Valeur de x : **6.0**.

5. **$x = (\text{float})p / q$;**

- Conversion de p en flottant : $(\text{float})p = 6.0$.
- Expression : $6.0 / 1 = 6.0$.
- Résultat : $x = 6.0$.
- Valeur de x : **6.0**.

6. **$x = (p + 0.5) / n$;**

- Ajout de 0.5 à p : $p + 0.5 = 6 + 0.5 = 6.5$.
- Expression : $6.5 / 1 = 6.5$.
- Résultat : $x = 6.5$.
- Valeur de x : **6.5**.

7. **$x = (\text{int})(p + 0.5) / n$;**

- Ajout de 0.5 à p : $p + 0.5 = 6.5$.
- Conversion en entier : $(\text{int})(6.5) = 6$.
- Expression : $6 / 1 = 6$.
- Résultat : $x = 6$.
- Valeur de x : **6.0** (converti en flottant à cause du type de x).

8. **$q = n * (p > n ? n : p)$;**

- L'opérateur ternaire ($? :$) fonctionne comme ceci :

- Si $p > n \rightarrow 6 > 1$, ce qui est **vrai**.
- Le résultat est donc n (car l'opérateur ternaire retourne n si la condition est vraie).
- Expression : $q = n * n = 1 * 1 = 1$.
- Résultat : $q = 1$.
- Valeur de q : **1**.